



Karol Mazur

Kierunek: informatyka

Specjalizacja: sztuczna inteligencja

Numer albumu: 394248

Algorytmy kwantowe dla wybranych problemów grafowych

Praca magisterska

wykonana pod kierunkiem

dr hab. Kordiana A. Smolińskiego

w Katedrze Fizyki Teoretycznej

WFiIS UŁ

Łódź 2026

Spis treści

1	Wstęp	4
2	Podstawy teoretyczne	5
2.1	Problem najkrótszej ścieżki	5
2.2	Algorytm Dijkstry	5
2.3	Znajdowanie minimum	7
2.3.1	Wersja naiwna	7
2.3.2	Wersja z kopcem binarnym	7
2.3.3	Wersja kwantowa	8
2.4	Rodzaje grafów	13
2.4.1	Graf gęsty	15
2.4.2	Graf o średniej gęstości	16
2.4.3	Graf rzadki	17
2.4.4	Przypadek szczególny	18
2.5	Reprezentacja grafów	19
3	Opis implementacji	21
3.1	Tworzenie grafów	21
3.1.1	Generowanie grafu gęstego	22
3.1.2	Generowanie grafu o średniej gęstości	22
3.1.3	Generowanie grafu rzadkiego	22
3.1.4	Generowanie przypadku szczególnego	23
3.2	Implementacja algorytmu Dijkstry	23
3.2.1	Wersja naiwna	23
3.2.2	Wersja z kopcem binarnym	24

Wstęp

Duża klasa problemów życia codziennego może być modelowanych przez odpowiadające im problemy grafowe. Wiele problemów grafowych jest NP-zupełnych lub NP-trudnych; są też problemy, dla których istnieją algorytmy wielomianowe. Dla wielu tych algorytmów kluczowym etapem jest przeprowadzenie wyszukiwania, np. minimalnej drogi. Ten etap można znacząco przyspieszyć wykorzystując kwantowy algorytm wyszukiwania minimum, będący uogólnieniem znanego kwantowego algorytmu wyszukiwania Grovera.

Celem tej pracy jest porównanie klasycznego i kwantowego wariantu algorytmu Dijkstry, służącego do znajdowania najkrótszej drogi w grafie. Algorytm ten zostanie zaimplementowany na trzy różne sposoby, a następnie zostanie sprawdzone, jak te podejścia radzą sobie na różnych rodzajach i rozmiarach grafów.

W pierwszej części pracy przedstawione zostaną podstawowe zagadnienia teoretyczne, obejmujące definicję grafów, ich rodzaje wykorzystywane w przeprowadzonych badaniach oraz zasadę działania algorytmu Dijkstry. Omówione zostaną również różne sposoby implementacji tego algorytmu. W dalszej części zaprezentowane zostaną wyniki przeprowadzonych eksperymentów wraz z ich analizą statystyczną, umożliwiającą ocenę rzeczywistej wydajności poszczególnych implementacji w zależności od charakterystyki analizowanych grafów. Na końcu pracy zamieszczono instrukcję instalacji i konfiguracji środowiska oraz wymaganych bibliotek, umożliwiającą samodzielne uruchomienie aplikacji i odtworzenie przedstawionych eksperymentów.

Podstawy teoretyczne

2.1 Problem najkrótszej ścieżki

Problem wyznaczania najkrótszej ścieżki należy do podstawowych zagadnień teorii grafów. Jego celem jest znalezienie ścieżki pomiędzy dwoma wierzchołkami grafu, której suma wag wszystkich krawędzi jest minimalna. Problem ten znajduje zastosowanie w wielu dziedzinach informatyki, takich jak systemy nawigacji, routing w sieciach komputerowych, logistyka, planowanie tras czy analiza sieci społecznościowych. Formalnie graf definiowany jest jako para [3, s. 2]

$$G = (V, E) \quad (2.1)$$

gdzie (V) oznacza zbiór wierzchołków, natomiast (E) zbiór krawędzi łączących pary wierzchołków. W przypadku grafów ważonych każdej krawędzi przypisana jest waga określająca koszt przejścia pomiędzy dwoma sąsiadującymi wierzchołkami. Jeżeli wszystkie wagi są nieujemne, jednym z najbardziej efektywnych algorytmów rozwiązujących problem najkrótszych ścieżek z początkowego wierzchołka jest algorytm Dijkstry.

2.2 Algorytm Dijkstry

Algorytm Dijkstry [4] został po raz pierwszy przedstawiony w 1959 roku przez Edsger W. Dijkstra. Jest to algorytm zachłanny, który znajduje najkrótsze ścieżki od jednego wierzchołka źródłowego do wszystkich pozostałych wierzchołków w grafie o nieujemnych wagach krawędzi. Podstawową ideą algorytmu jest stopniowe powiększanie zbioru odwiedzonych wierzchołków, dla których wyznaczono już najkrótszą możliwą odległość od źródła. W każdej kolejnej iteracji wybierany jest wierzchołek o najmniejszej znanej odległości spośród nieodwiedzonych, po czym wykonywana jest operacja relaksacji wszystkich wychodzących z niego krawędzi.

Relaksacja polega na sprawdzeniu, czy przejście przez aktualnie analizowany wierzchołek prowadzi do krótszej ścieżki do sąsiada. Jeżeli zachodzi zależność [2, s. 649]

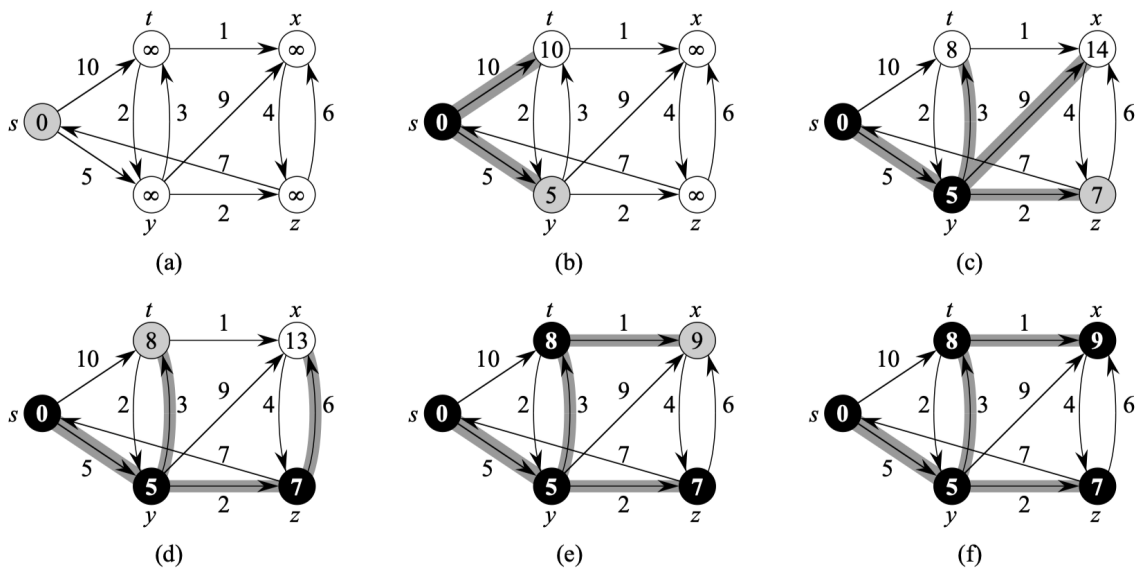
$$d(v) > d(u) + w(u, v) \quad (2.2)$$

wartość ($d(v)$) zostaje zaktualizowana. Algorytm kończy działanie po odwiedzeniu wszystkich osiągalnych wierzchołków.

Algorytm 1 Algorytm Dijkstry

1. Zainicjalizuj odległości wszystkich wierzchołków od wierzchołka źródłowego.
2. Utwórz pusty zbiór odwiedzonych wierzchołków $S \leftarrow \emptyset$.
3. Umieść wszystkie wierzchołki grafu w zbiorze Q .
4. Dopóki zbiór Q nie jest pusty, wykonuj:
 - (a) Wybierz i usuń ze zbioru Q wierzchołek u o najmniejszej aktualnej odległości od źródła.
 - (b) Dodaj wierzchołek u do zbioru odwiedzonych S .
 - (c) Dla każdego sąsiada v wierzchołka u wykonaj relaksację krawędzi (u, v) .

Opracowanie własne na podstawie [2, s. 658]



Rysunek 2.1: Przykład działania algorytmu Dijkstry. [2, s. 659]

2.3 Znajdowanie minimum

Najbardziej kosztownym etapem algorytmu Dijkstry jest wyznaczenie kolejnego nieodwiedzonego wierzchołka o najmniejszej odległości. Sposób w jaki jest to zaimplementowane znacząco wpływa na złożoność całego algorytmu Dijkstry. Dlatego w celu porównania różnych podejść w niniejszej pracy wykorzystano trzy wersje algorytmu Dijkstry z różnymi sposobami znajdowania minimum:

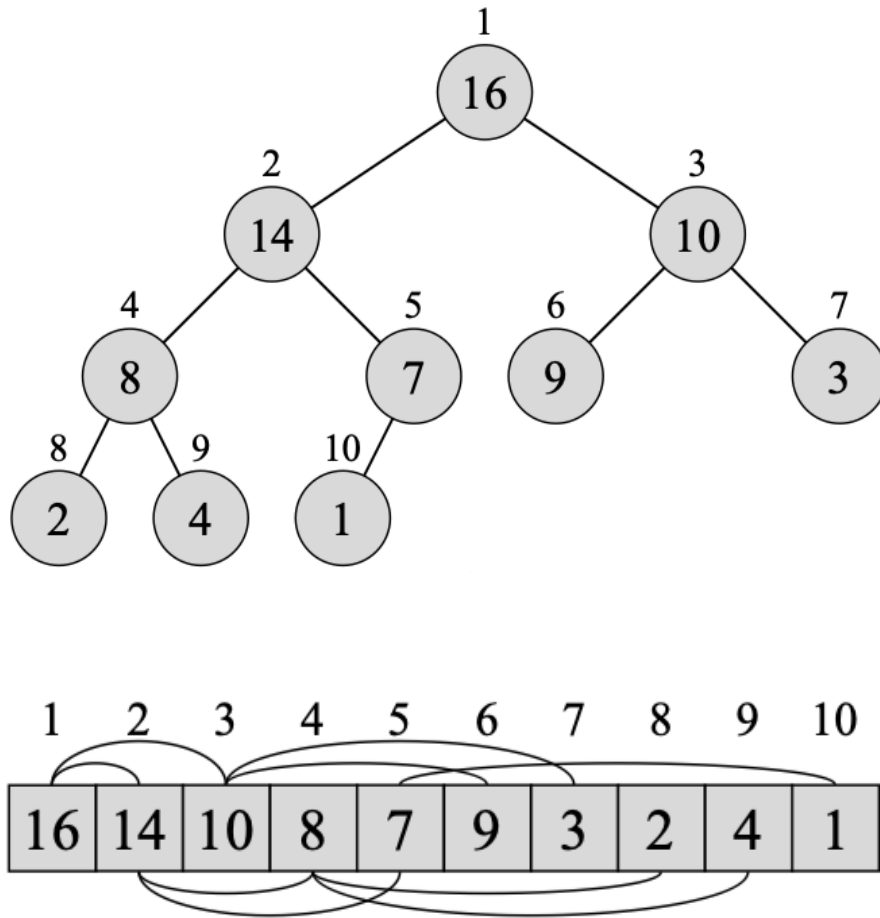
- wersję naiwną,
- wersję z wykorzystaniem kopca binarnego,
- wersję wykorzystującą kwantowy algorytm wyszukiwania minimum.

2.3.1 Wersja naiwna

W wersji naiwnej wybór kolejnego wierzchołka realizowany jest poprzez liniowe przeszukanie tablicy odległości w celu znalezienia najmniejszej wartości spośród wszystkich nieodwiedzonych wierzchołków. Operacja ta jest wykonywana w każdej iteracji algorytmu Dijkstry, a ponieważ liczba iteracji jest proporcjonalna do liczby wierzchołków V , całkowity koszt wyszukiwania minimum wynosi $O(V^2)$. Pomimo niskiej wydajności, wersja naiwna cechuje się prostotą implementacji i często wykorzystywana jest jako punkt odniesienia podczas porównywania bardziej zaawansowanych wariantów algorytmu jak na przykład kopiec binarny.

2.3.2 Wersja z kopcem binarnym

W celu przyspieszenia operacji wyboru kolejnego wierzchołka można zastosować kopiec binarny przechowujący pary składające się z identyfikatora wierzchołka oraz aktualnej wartości odległości. Kopiec binarny jest strukturą danych reprezentowaną w postaci niemal pełnego drzewa binarnego, w którym każdy węzeł posiada co najwyżej dwoje dzieci. W kopcu minimalnym spełniona jest własność, zgodnie z którą wartość klucza każdego węzła jest nie większa od wartości kluczy jego potomków. Oznacza to, że element o najmniejszym kluczu znajduje się zawsze w korzeniu drzewa. Jego pobranie oraz usunięcie wymaga czasu $O(\log V)$. Wstawienie nowego elementu lub aktualizacja wartości klucza również wykonywane są w czasie $O(\log V)$ poprzez odpowiednie odtworzenie własności kopca. Dzięki zastosowaniu kopca binarnego wyszukiwanie wierzchołka o najmniejszej odległości nie wymaga liniowego przeszukiwania wszystkich elementów [2, r. 6].



Rysunek 2.2: Przykład kopca binarnego *max* wraz z listą jego wartości i kluczy [2, s. 151]. W niniejszej pracy do implementacji Dijkstry wykorzystywana jest wersja *min* kopca binarnego

2.3.3 Wersja kwantowa

W implementacji kwantowej klasyczna operacja wyboru nieodwiedzonego wierzchołka o najmniejszej wartości odległości została zastąpiona kwantowym algorytmem wyszukiwania minimum stworzonym przez Dürra i Høyer [5]. Algorytm ten stanowi rozwinięcie kwantowego algorytmu wyszukiwania Grovera [6], wykorzystując uogólniony algorytm wyszukiwania opracowany przez Boyera, Brassarda, Høyer i Tappa (BBHT) [7]. W przeciwieństwie do klasycznego algorytmu Grovera, który zakłada znajomość liczby rozwiązań, algorytm ten umożliwia skuteczne wyszukiwanie również w przypadku nieznaney liczby rozwiązań.

Algorytm Dürra–Høyer realizuje wyszukiwanie minimum poprzez iteracyjne wywoływanie algorytmu BBHT w celu znalezienia elementu o wartości mniejszej od aktualnego kandydata na minimum. Po znalezieniu takiego elementu zostaje on nowym kandydatem, a proces jest powtarzany aż do momentu, gdy z dużym prawdopodobieństwem nie istnieje już element o mniejszej wartości.

Algorytm Grovera

Algorytm Grovera to kwantowa procedura sprawdzania obecności klucza w bazie danych. W nim baza danych nie ma ustalonej struktury jak w klasycznych bazach gdzie dane są zorganizowane np. w struktury drzewiaste. Jego celem jest znalezienie elementu/elementów z wykorzystaniem wyroczni (*oracle*), która potrafi rozpoznać poprawne rozwiązania. W przeciwieństwie do klasycznego przeszukiwania liniowego wymagającego średnio $O(N)$ sprawdzeń, algorytm Grovera osiąga złożoność $O(\sqrt{N})$ [6] .

Ważnym ograniczeniem podstawowej wersji algorytmu jest konieczność wykonania odpowiedniej liczby iteracji. Liczba ta zależy od liczby elementów spełniających zadany warunek określony przez wyrocznię i w ogólnym przypadku nie jest znana. Zbyt mała liczba iteracji nie zapewnia wystarczającego wzmocnienia amplitudy rozwiązania, natomiast zbyt duża powoduje jej ponowne zmniejszenie, co obniża prawdopodobieństwo poprawnego wyniku. Problem ten został rozwiązany w algorytmie Boyera, Brassarda, Høyer'a i Tappa (BBHT), który zostanie omówiony w kolejnym podrozdziale.

Algorytm 2 Algorytm Grovera

1. Przygotuj rejestr kwantowy w stanie równomiernej superpozycji wszystkich N możliwych stanów.
2. Powtarzaj operator Grovera odpowiednią liczbę razy:
 - (a) Zastosuj wyrocznię (*oracle*), która zmienia znak amplitudy stanów odpowiadających rozwiązaniom problemu.
 - (b) Zastosuj operator dyfuzji (*diffusion transform*), który odbija amplitudy wszystkich stanów względem ich średniej wartości, zwiększając amplitudę stanów oznaczonych przez wyrocznię i zmniejszając amplitudy pozostałych.
3. Wykonaj pomiar rejestru kwantowego.
4. Zwróć zmierzony stan jako wynik wyszukiwania.

Opracowanie własne na podstawie [6]

W pierwszym kroku tworzona jest superpozycja wszystkich możliwych stanów. Żeby to uzyskać w rejestrze kwantowym złożonym z n kubitów stosujemy n -krotnie bramkę Hadamarda. [8, s. 162, wz. (4.66)]

$$|\psi\rangle = H^{\otimes n}|00\cdots 0\rangle = (H|0\rangle)^{\otimes n} = \left(\frac{|0\rangle + |1\rangle}{\sqrt{2}}\right)^{\otimes n} = \frac{1}{\sqrt{N}} \sum_{i=0}^{N-1} |i\rangle. \quad (2.3)$$

Dla stanu kubitów złożonego z $n=3$ kubitów odpowiada to temu zapisowi [8, s. 278, wz. (5.366)]

$$|\psi'\rangle = \frac{1}{2\sqrt{2}}(|000\rangle + |001\rangle + |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle). \quad (2.4)$$

Wszystkie amplitudy prawdopodobieństwa mają wartość $1/\sqrt{N}$ co oznacza że każdy stan jest jednakowo prawdopodobny w przypadku pomiaru.

Kolejny etap stanowi iteracyjne wykonywanie operatora Grovera. Każda iteracja składa się z dwóch operacji. Pierwszą z nich jest działanie wyrocni, która identyfikuje stany spełniające zadany warunek poprzez odwrócenie znaku ich amplitudy. Jest to funkcja która odpowiada za wskazanie szukanego elementu [8, s. 162, wz. (4.67)].

$$f(i) = \begin{cases} 1, & \text{jeśli } i \text{ reprezentuje szukany element,} \\ 0, & \text{w przeciwnym przypadku.} \end{cases} \quad (2.5)$$

Oczekuje się, że działanie tej funkcji implementowanej przez unitarny operator U_f można opisać wzorem [8, s. 162, wz. (4.68)]

$$U_f(|i\rangle|j\rangle) = |i\rangle|j \oplus f(i)\rangle. \quad (2.6)$$

Przypadek w którym funkcja ta zmienia znak na rejestrze kwantowym przy trzeciej amplitudzie można zapisać w ten sposób [8, s. 278, wz. (5.367)]

$$|\psi''\rangle = \frac{1}{2\sqrt{2}}(|000\rangle + |001\rangle - |010\rangle + |011\rangle + |100\rangle + |101\rangle + |110\rangle + |111\rangle). \quad (2.7)$$

Wykonanie pomiaru bezpośrednio po zastosowaniu wyrocni nie pozwala jeszcze na wskazanie poprawnego rozwiązania. Wynika to z faktu, że wyrocnia zmienia jedynie znak amplitudy stanu odpowiadającego szukanemu elementowi, nie zmieniając jej wartości bezwzględnej. Podczas pomiaru wyznaczane jest natomiast prawdopodobieństwo otrzymania danego stanu, które jest równe kwadratowi modułu jego amplitudy. W rezultacie informacja o znaku amplitudy zostaje utracona. Z tego powodu konieczne jest wykonanie kolejnego etapu algorytmu – operatora dyfuzji nazywanego również operatorem obrotu wokół średniej. Jego zadaniem jest zwiększenie amplitud stanów oznaczonych przez wyrocnię kosztem amplitud pozostałych stanów.

Definicja operatora obrotu wokół średniej, oznaczonego jako A przybiera postać [8, s. 163, wz. (4.71)]

$$A = 2|\psi\rangle\langle\psi| - I. \quad (2.8)$$

gdzie $|\psi\rangle$ to stan opisany wzorem (2.3)

Po zastosowaniu operatora U_f otrzymuje się stan [8, s. 163, wz. (4.72)]

$$|\psi_1\rangle = |\psi\rangle - \frac{2}{\sqrt{2^n}}|i_{U_f}\rangle. \quad (2.9)$$

Odpowiada to takiej bramce kwantowej przedstawionej w formie macierzy [8, s. 277, wz. (5.364)]

$$G_{L \times L} = \begin{pmatrix} \frac{2}{L} - 1 & \frac{2}{L} & \cdots & \frac{2}{L} \\ \frac{2}{L} & \frac{2}{L} - 1 & \cdots & \frac{2}{L} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{2}{L} & \frac{2}{L} & \cdots & \frac{2}{L} - 1 \end{pmatrix}, \quad L = 2^n. \quad (2.10)$$

Całość działania operatora obrotu wokół średniej G na stanie (2.7) można przedstawić w ten sposób [8, s. 278, wz. (5.368)]

$$|\psi'''\rangle = G|\psi''\rangle = \begin{pmatrix} -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} & \frac{1}{4} \\ \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & \frac{1}{4} & -\frac{3}{4} \end{pmatrix} \begin{pmatrix} \frac{1}{2\sqrt{2}} \\ \frac{1}{2\sqrt{2}} \\ -\frac{1}{2\sqrt{2}} \\ \frac{1}{2\sqrt{2}} \\ \frac{1}{2\sqrt{2}} \\ \frac{1}{2\sqrt{2}} \\ \frac{1}{2\sqrt{2}} \\ \frac{1}{2\sqrt{2}} \end{pmatrix} = \begin{pmatrix} \frac{1}{4\sqrt{2}} \\ \frac{1}{4\sqrt{2}} \\ \frac{5}{4\sqrt{2}} \\ \frac{1}{4\sqrt{2}} \\ \frac{1}{4\sqrt{2}} \\ \frac{1}{4\sqrt{2}} \\ \frac{1}{4\sqrt{2}} \\ \frac{1}{4\sqrt{2}} \end{pmatrix}. \quad (2.11)$$

Powtarzanie tych dwóch operacji (wyroczni i dyfuzji) prowadzi do stopniowego zwiększania prawdopodobieństwa uzyskania poprawnego rozwiązania. Po wykonaniu odpowiedniej liczby iteracji przeprowadzany jest pomiar rejestru kwantowego, podczas którego prawdopodobieństwo uzyskania danego stanu jest równe kwadratowi modułu jego amplitudy. Dzięki wcześniejszemu wzmocnieniu amplitudy stanu odpowiadającego rozwiązaniu zostaje on uzyskany z wysokim prawdopodobieństwem.

Algorytm BBHT

Algorytm opracowany przez Boyera, Brassarda, Høyer i Tappa stanowi rozszerzenie algorytmu Grovera przeznaczone dla sytuacji, w której liczba rozwiązań problemu nie jest znana. W przeciwieństwie do klasycznej wersji algorytmu nie można z góry określić optymalnej liczby iteracji operatora Grovera.

Zamiast tego algorytm stopniowo zwiększa zakres możliwej liczby iteracji.[7]

Algorytm 3 Algorytm BBHT

1. Zainicjalizuj parametry algorytmu: ustaw $m = 1$ oraz wybierz współczynnik $\lambda = \frac{6}{5}$ (ogólnie $\lambda \in (1, \frac{4}{3})$).
2. Wylosuj liczbę całkowitą j równomiernie z przedziału $\{0, 1, \dots, m - 1\}$.
3. Przygotuj rejestr kwantowy w stanie superpozycji wszystkich N możliwych stanów.
4. Wykonaj j iteracji algorytmu Grovera (Alg. 2).
5. Wykonaj pomiar rejestru kwantowego i otrzymaj stan i .
6. Jeżeli $T[i] = x$, zakończ działanie algorytmu i zwróć znalezione rozwiązanie.
7. W przeciwnym razie zwiększ wartość parametru m zgodnie z zależnością

$$m = \min(\lambda m, \sqrt{N}),$$

a następnie wróć do kroku 2.

Opracowanie własne na podstawie [7]

Losowy dobór liczby iteracji pozwala uniknąć sytuacji, w której zastosowanie zbyt dużej jej ilości zmniejszyłoby prawdopodobieństwo uzyskania poprawnego rozwiązania. Dzięki stopniowemu zwiększaniu parametru m algorytm zachowuje kwadratowe przyspieszenie względem algorytmów klasycznych, osiągając oczekiwany czas działania rzędu

$$O\left(\sqrt{\frac{N}{t}}\right) \tag{2.12}$$

gdzie t oznacza liczbę rozwiązań problemu [7].

Kwantowy algorytm wyszukiwania minimum

Algorytm wyszukiwania minimum, zaproponowany przez Dürra i Høyera, wykorzystuje algorytm BBHT jako podprocedurę służącą do znajdowania elementów o wartości mniejszej od aktualnie przyjętego progu. Algorytm ten iteracyjnie aktualizuje indeks elementu o najmniejszej dotychczas znalezionej wartości. Dzięki

zastosowaniu kwantowego przeszukiwania możliwe jest znalezienie minimum tablicy w oczekiwanym czasie rzędu $\mathcal{O}(\sqrt{N})$ przy prawdopodobieństwie sukcesu wynoszącym co najmniej $\frac{1}{2}$ [5].

Algorytm 4 Kwantowy algorytm wyszukiwania minimum

1. Wylosuj indeks y z przedziału $\{0, 1, \dots, N - 1\}$ i przyjmij go jako początkowy próg.
2. Powtarzaj poniższe kroki do momentu przekroczenia limitu czasu działania opisanego w (2.13):
 - (a) Przygotuj rejestr kwantowy w stanie równomiernej superpozycji wszystkich stanów.
 - (b) Oznacz wszystkie stany odpowiadające indeksom j , dla których zachodzi

$$T[j] < T[y].$$

- (c) Uruchom algorytm BBHT (Alg. 3) w celu znalezienia jednego z oznaczonych elementów.
- (d) Wykonaj pomiar rejestru kwantowego i otrzymaj indeks y' .
- (e) Jeżeli

$$T[y'] < T[y],$$

to zaktualizuj próg, przyjmując $y \leftarrow y'$.

3. Po zakończeniu iteracji zwróć indeks y jako wynik działania algorytmu.

Opracowanie własne na podstawie [5]

Łączny oczekiwany czas działania algorytmu do momentu, w którym zmienna y będzie wskazywać indeks elementu minimalnego, wynosi[5]

$$m_0 = \frac{45}{4}\sqrt{N} + \frac{7}{10}\lg^2 N. \quad (2.13)$$

2.4 Rodzaje grafów

Wpływ rodzaju grafu na wydajność algorytmu jest jednym z najważniejszych aspektów niniejszej pracy. Analiza zostanie przeprowadzona na czterech różnych rodzajach grafów: grafie rzadkim, średnio gęstym, gęstym oraz specjalnie skonstruowanym grafie, który dzięki odpowiednio dobranym wagom i liczbie krawędzi wymusza maksymalną liczbę relaksacji. We wszystkich analizowanych przypadkach rozpatrywane są

grafy nieskierowane, bez pętli własnych, o nieujemnych wagach krawędzi. W przypadku tego ostatniego rodzaju grafu wagi krawędzi będą ściśle określone. Natomiast dla pozostałych grafów wagi będą losowane z przedziału 1 do n^2 , gdzie n oznacza liczbę wierzchołków.

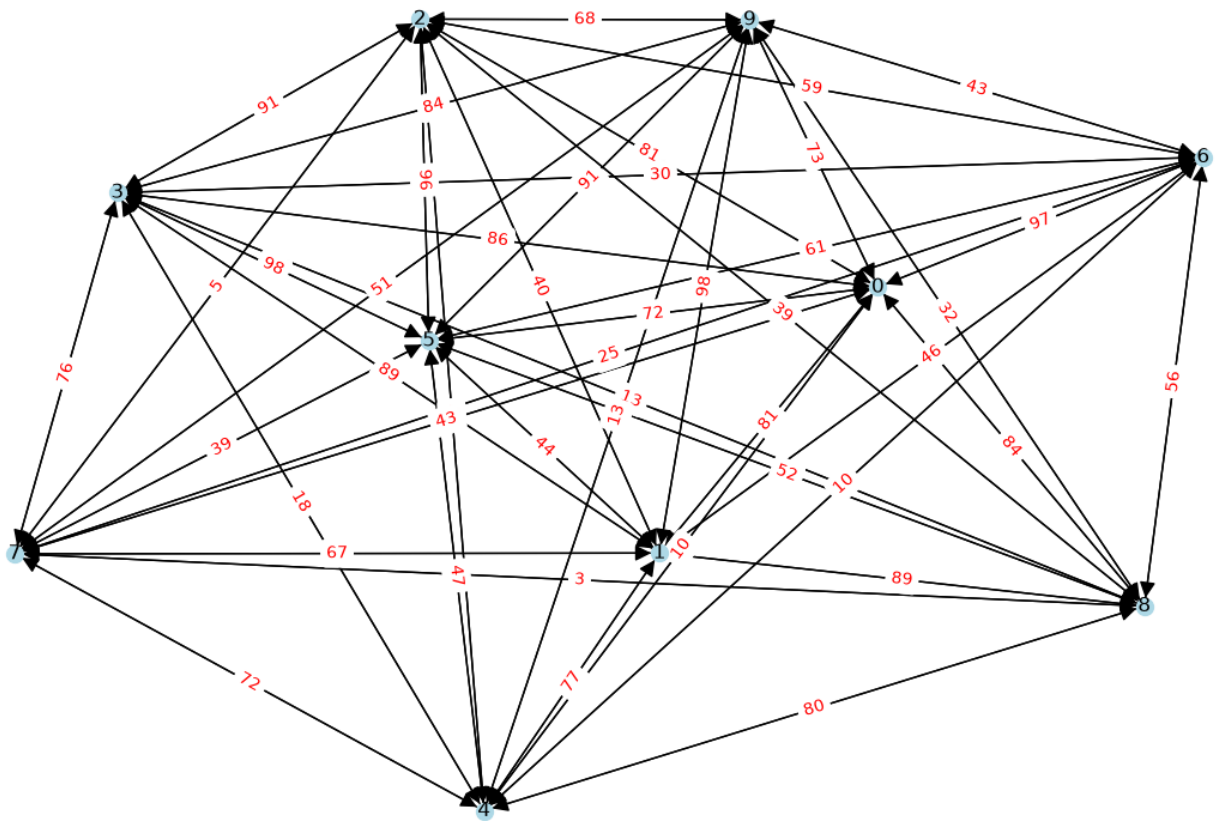
2.4.1 Graf gęsty

Graf gęsty zawiera liczbę krawędzi bliską maksymalnej [1]

$$|E| = O(|V|^2).$$

W niniejszej pracy graf gęsty jest grafem pełnym nieskierowanym, w którym liczba krawędzi jest równa maksymalnej liczbie krawędzi dopuszczalnej dla grafu nieskierowanego bez pętli [9]

$$E = \frac{V(V - 1)}{2}.$$



Rysunek 2.3: Przykład grafu "dense" analizowanego w pracy

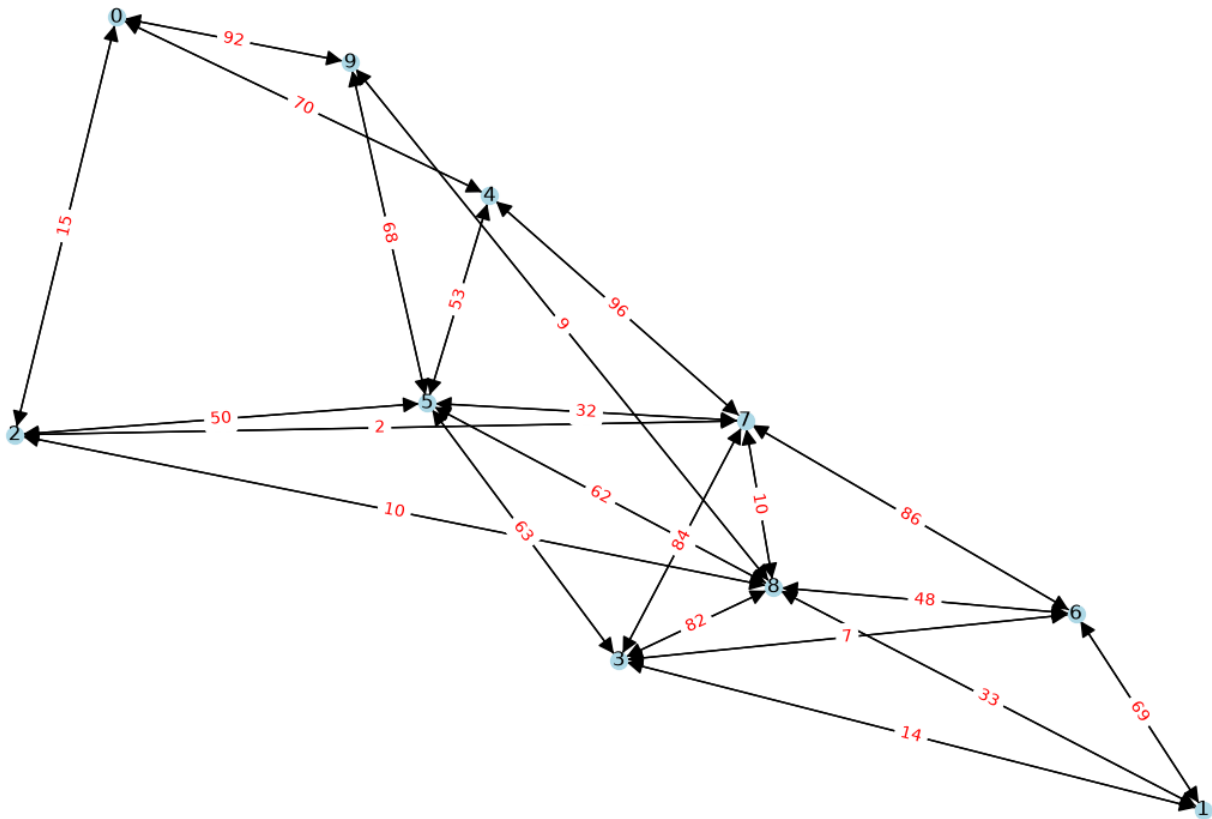
2.4.2 Graf o średniej gęstości

Graf średnio gęsty zajmuje pośrednią pozycję pomiędzy grafami rzadkimi i pełnymi. Liczba krawędzi jest znacznie większa niż liczba wierzchołków, jednak nadal nie osiąga wartości maksymalnej jak w grafach gęstych [1]

$$|E| = O(|V|^2).$$

W niniejszej pracy jako graf o średniej gęstości przyjęto graf nieskierowany, którego liczba krawędzi stanowi połowę maksymalnej liczby krawędzi

$$E = \frac{V(V-1)}{4}.$$



Rysunek 2.4: Przykład grafu "half-edges" analizowanego w pracy

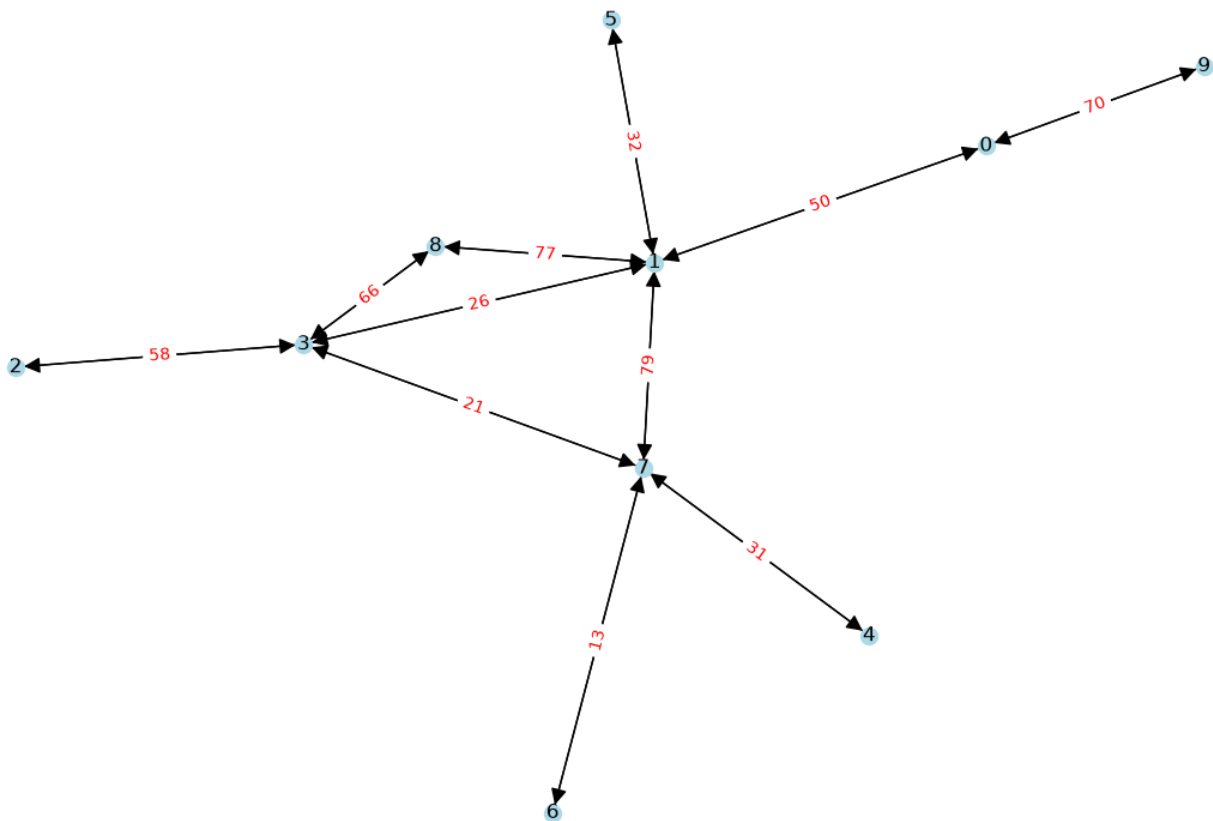
2.4.3 Graf rzadki

Graf rzadki charakteryzuje się niewielką liczbą krawędzi względem maksymalnej liczby możliwych połączeń [1]

$$|E| = O(|V|).$$

Przykładami grafów rzadkich są sieci drogowe, drzewa oraz wiele rzeczywistych sieci komputerowych. W niniejszej pracy jako graf rzadki przyjęto graf nieskierowany, w którym liczba krawędzi jest równa liczbie wierzchołków

$$E = V.$$



Rysunek 2.5: Przykład grafu "sparse" analizowanego w pracy

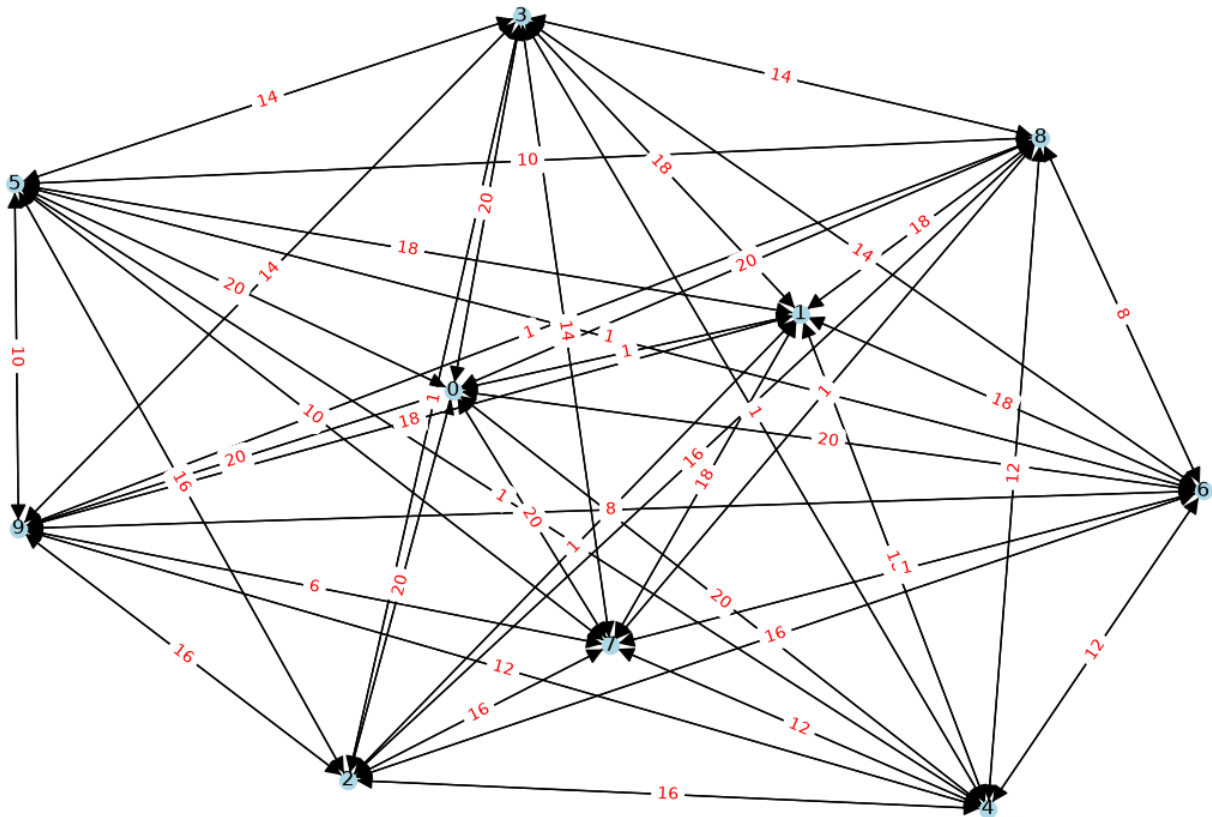
2.4.4 Przypadek szczególny

Przypadek szczególny jest specjalnie przygotowanym grafem pełnym. W odróżnieniu od pozostałych analizowanych grafów jego struktura oraz wagi krawędzi nie są losowe, lecz zostały dobrane w taki sposób, aby wymusić dużą liczbę operacji relaksacji wykonywanych przez algorytm Dijkstry. Tak jak graf gęsty, przypadek szczególny zawiera maksymalną liczbę krawędzi dla grafu nieskierowanego bez pętli [9]

$$E = \frac{V(V-1)}{2}.$$

Wagi krawędzi są wyznaczone zgodnie z następującą zależnością

$$w(i, j) = \begin{cases} 1, & \text{gdy } j = i + 1, \\ 2(|V| - i), & \text{gdy } j > i + 1. \end{cases}$$



Rysunek 2.6: Przykład grafu "special case" analizowanego w pracy

2.5 Reprezentacja grafów

Graf może być reprezentowany na różne sposoby, przy czym do najczęściej stosowanych metod należą macierz sąsiedztwa oraz lista sąsiedztwa. W celu zilustrowania obu sposobów reprezentacji wykorzystano graf przedstawiony na rysunku 2.4.

$$A = \begin{pmatrix} 0 & 0 & 15 & 0 & 70 & 0 & 0 & 0 & 0 & 92 \\ 0 & 0 & 0 & 14 & 0 & 0 & 69 & 0 & 33 & 0 \\ 15 & 0 & 0 & 0 & 0 & 50 & 0 & 2 & 10 & 0 \\ 0 & 14 & 0 & 0 & 0 & 63 & 7 & 84 & 82 & 0 \\ 70 & 0 & 0 & 0 & 0 & 53 & 0 & 96 & 0 & 0 \\ 0 & 0 & 50 & 63 & 53 & 0 & 0 & 32 & 62 & 68 \\ 0 & 69 & 0 & 7 & 0 & 0 & 0 & 86 & 48 & 0 \\ 0 & 0 & 2 & 84 & 96 & 32 & 86 & 0 & 10 & 0 \\ 0 & 33 & 10 & 82 & 0 & 62 & 48 & 10 & 0 & 9 \\ 92 & 0 & 0 & 0 & 0 & 68 & 0 & 0 & 9 & 0 \end{pmatrix} \quad (2.14)$$

Macierz sąsiedztwa przedstawia graf w postaci dwuwymiarowej tablicy, w której element znajdujący się na przecięciu i -tego wiersza i j -tej kolumny określa istnienie krawędzi pomiędzy odpowiadającymi im wierzchołkami lub przechowuje jej wagę. Reprezentacja ta zawiera informacje o wszystkich możliwych parach wierzchołków, niezależnie czy są one połączone krawędzią.

$$\begin{aligned} 0 &: \{(2, 15), (9, 92), (4, 70)\}, \\ 1 &: \{(6, 69), (8, 33), (3, 14)\}, \\ 2 &: \{(8, 10), (0, 15), (7, 2), (5, 50)\}, \\ 3 &: \{(5, 63), (7, 84), (8, 82), (1, 14), (6, 7)\}, \\ 4 &: \{(7, 96), (5, 53), (0, 70)\}, \\ 5 &: \{(7, 32), (8, 62), (3, 63), (9, 68), (4, 53), (2, 50)\}, \\ 6 &: \{(8, 48), (1, 69), (7, 86), (3, 7)\}, \\ 7 &: \{(4, 96), (6, 86), (5, 32), (2, 2), (8, 10), (3, 84)\}, \\ 8 &: \{(2, 10), (6, 48), (1, 33), (9, 9), (5, 62), (7, 10), (3, 82)\}, \\ 9 &: \{(8, 9), (5, 68), (0, 92)\}. \end{aligned} \quad (2.15)$$

W implementacji przedstawionej w niniejszej pracy graf został zapisany w postaci listy sąsiedztwa. W tym sposobie reprezentacji grafu dla każdego wierzchołka przechowywana jest lista wszystkich wierzchołków z nim połączonych wraz z odpowiadającymi im wagami krawędzi. W przypadku grafu nieskierowanego każda krawędź występuje dwukrotnie – na liście sąsiedztwa obu wierzchołków, które łączy.

Opis implementacji

W niniejszym rozdziale przedstawiono sposób implementacji rozwiązania opisanego od strony teoretycznej w poprzednim rozdziale. Implementacja obejmuje generowanie grafów, trzy warianty algorytmu Dijkstry, analizę otrzymanych wyników oraz tworzenie wykresów umożliwiających ich porównanie.

Poszczególne etapy działania programu zostały rozdzielone, żeby mogły być uruchamiane niezależnie. Dane pomiędzy nimi przekazywane są przy użyciu plików JSON. Wygenerowane grafy są zapisywane do plików, a następnie odczytywane przez moduły wykonujące odpowiednie wersje algorytmu Dijkstry. Wyniki zapisywane są w plikach JSON, które następnie wykorzystywane są przez moduły do analizy statystycznej. Jej wyniki stanowią z kolei dane wejściowe dla modułów odpowiedzialnych za generowanie wykresów. Cały proces przetwarzania danych w implementacji można przedstawić w następujący sposób:

generowanie grafów → zapis do JSON → wykonanie algorytmów → zapis wyników do JSON → analiza
statystyczna → zapis statystyk do JSON → generowanie wykresów.

Taki podział na niezależne moduły ułatwia przeprowadzanie eksperymentów z wykorzystaniem poszczególnych wersji algorytmu Dijkstry, analizę uzyskanych wyników oraz ponowne generowanie wykresów bez konieczności wykonywania całego procesu od początku.

3.1 Tworzenie grafów

Pierwszym etapem jest przygotowanie grafów stanowiących dane wejściowe dla wszystkich analizowanych wariantów algorytmu. W części teoretycznej przedstawiono cztery rodzaje grafów wykorzystywane w pracy: graf rzadki, graf o średniej gęstości, graf gęsty oraz przypadek szczególny. Ich właściwości, a także przyjęte liczby krawędzi, opisano w rozdziale dotyczącym rodzajów grafów 2.4.

Liczba wierzchołków grafów oraz liczba różnych grafów generowanych dla każdego badanego rozmiaru są określane za pomocą pliku konfiguracyjnego. Pozwala to na łatwe dostosowanie parametrów bez konieczności modyfikowania kodu źródłowego.

Grafy przechowywane są w postaci listy sąsiedztwa, zgodnie z reprezentacją opisaną wcześniej w rozdziale 2.5. Po wygenerowaniu każdy graf jest zapisywany do pliku JSON. Jego nazwa zawiera informacje pozwalające określić typ grafu, jego rozmiar oraz numer. Dzięki temu wszystkie implementacje algorytmu Dijkstry mogą korzystać z tych samych danych wejściowych.

3.1.1 Generowanie grafu gęstego

Graf gęsty został zaimplementowany jako graf pełny. Oznacza to, że dla każdej pary różnych wierzchołków tworzona jest dokładnie jedna krawędź. Liczba krawędzi jest zatem zgodna z zależnością

$$|E| = \frac{|V|(|V| - 1)}{2}, \quad (3.1)$$

przedstawioną również w części teoretycznej w rozdziale 2.4.1. Dla każdej utworzonej krawędzi losowana jest jej waga. Przykład grafu gęstego przedstawiono na rys 2.3.

3.1.2 Generowanie grafu o średniej gęstości

W przypadku grafu o średniej gęstości liczba tworzonych krawędzi stanowi połowę maksymalnej liczby krawędzi grafu pełnego, zgodnie z zależnością przedstawioną wcześniej w rozdziale 2.4.2

$$|E| = \frac{|V|(|V| - 1)}{4}. \quad (3.2)$$

Kolejne krawędzie tworzone są pomiędzy losowo wybranymi parami różnych wierzchołków. Podczas generowania sprawdzane jest, czy dana krawędź nie została utworzona wcześniej, dzięki czemu graf nie zawiera wielokrotnych krawędzi pomiędzy tą samą parą wierzchołków. Wagi krawędzi losowane są z zakresu określonego w części teoretycznej. Przykład wykorzystywanego grafu o średniej gęstości przedstawiono na rys 2.4.

3.1.3 Generowanie grafu rzadkiego

Graf rzadki generowany jest zgodnie z definicją przyjętą w części teoretycznej w rozdziale 2.4.3, zgodnie z którą liczba krawędzi jest równa liczbie wierzchołków

$$|E| = |V|. \quad (3.3)$$

Mechanizm losowania krawędzi oraz ich wag jest analogiczny jak w przypadku grafu o średniej gęstości. Różnica polega na większej liczbie tworzonych połączeń. Przykład grafu tego typu przedstawiono na rys 2.5.

3.1.4 Generowanie przypadku szczególnego

Ostatnim generowanym typem jest przypadek szczególny przedstawiony na rys 2.6. Podobnie jak graf gęsty jest on grafem pełnym, jednak wagi jego krawędzi nie są losowane. Sposób ich wyznaczania odpowiada zależności przedstawionej w części teoretycznej w rozdziale 2.4.4.

$$w(i, j) = \begin{cases} 1, & \text{gdy } j = i + 1, \\ 2(|V| - i), & \text{gdy } j > i + 1. \end{cases}$$

3.2 Implementacja algorytmu Dijkstry

Podstawowy sposób działania algorytmu Dijkstry został przedstawiony w alg. 1. Wszystkie trzy implementacje użyte w części działają według tego samego schematu. Różnią się przede wszystkim sposobem wykonania operacji wyboru nieodwiedzzonego wierzchołka o najmniejszej aktualnie znanej odległości.

W celu porównania różnych metod wyszukiwania minimum zaimplementowano trzy warianty:

- wersję naiwną,
- wersję wykorzystującą kopiec binarny,
- wersję wykorzystującą kwantowy algorytm wyszukiwania minimum.

3.2.1 Wersja naiwna

W wersji naiwnej wyszukiwanie minimum realizowane poprzez liniowe przeglądanie wszystkich wierzchołków grafu. W każdej iteracji algorytmu Dijkstry wybierany jest spośród nieprzetworzonych wierzchołków ten, którego aktualnie znana odległość od wierzchołka źródłowego jest najmniejsza. Sposób ten odpowiada wersji naiwnej opisanej w części teoretycznej w rozdziale 2.3.1.

Na potrzeby zliczania operacji podczas wyszukiwania wprowadzono zmienną przechowującą liczbę wykonanych operacji. Za pojedynczą operację przyjęto sprawdzenie jednego wierzchołka w wewnętrznej pętli wyszukiwania. Licznik ten jest zwiększany o 1 dla każdego sprawdzanego wierzchołka, niezależnie od tego,

czy zostanie on nowym kandydatem na minimum. Ponieważ podczas pojedynczego wyszukiwania przeglądane jest zawsze n wierzchołków, liczba zliczonych operacji wynosi

$$C_{\text{search}} = n. \quad (3.4)$$

Wyszukiwanie wykonywane jest n razy, ponieważ zewnętrzna pętla algorytmu również wykonuje n iteracji. Całkowita liczba operacji zliczonych dla jednego uruchomienia wersji naiwnej wynosi więc

$$C_{\text{naive}} = n^2. \quad (3.5)$$

Po zakończeniu działania algorytmu wartość zmiennej liczącej liczbę operacji jest zapisywana, a następnie przekazywana w pliku JSON do dalszej analizy statystycznej.

3.2.2 Wersja z kopcem binarnym

Druga klasyczna implementacja wykorzystuje kopiec binarny opisany w części teoretycznej w rozdziale 2.3.1. Przykładową strukturę kopca przedstawiono na rys. 2.2. W przeciwieństwie do wersji naiwnej wybór wierzchołka o najmniejszej odległości nie wymaga liniowego przeglądania wszystkich nieodwiedzonych wierzchołków.

Na potrzeby algorytmu zaimplementowano własną strukturę kopca, która zlicza wykonywane podczas jego działania operacje. Uwzględniane są ich dwa rodzaje:

- porównania wartości priorytetów,
- zamiany elementów wewnątrz kopca.

Operacje te są zliczane podczas wstawiania elementów do kopca oraz pobierania elementu o najmniejszym priorytecie. Znalezienie krótszej drogi podczas relaksacji powoduje aktualizację odległości danego wierzchołka i dodanie nowej wartości do kopca. Wstawienie elementu może wymagać ponownego uporządkowania kopca, co wiąże się z kolejnymi porównaniami i zamianami elementów. Całkowita liczba operacji dla wersji z kopcem jest sumą liczników obu operacji. Po zakończeniu algorytmu suma ta jest zapisywana do pliku JSON.

Bibliografia

- [1] B. Bollobas, O. Riordan. Metrics for sparse graphs. Surveys in Combinatorics 2009, LMS Lecture Notes Series 365, CUP 2009, pp. 211–287, 2008. <https://arxiv.org/abs/0708.1919>.
- [2] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, 3 edition, 2009. <https://www.cs.mcgill.ca/~akroitt/math/compsci/Cormen%20Introduction%20to%20Algorithms.pdf>.
- [3] Reinhard Diestel. *Graph Theory*. Springer-Verlag Berlin and Heidelberg GmbH & Co. KG, 5 edition, 2017. https://daiwz.net/course/disc_math/2023/Diestel_Graph_Theory.pdf.
- [4] Dijkstra, E.W. A note on two problems in connexion with graphs. Numer. Math. 1, 269–271, 1959. <https://ir.cwi.nl/pub/9256/9256D.pdf>.
- [5] Christoph Dürr and Peter Høyer. A quantum algorithm for finding the minimum. <https://arxiv.org/abs/quant-ph/9607014>, 1996.
- [6] Lov K. Grover. A fast quantum mechanical algorithm for database search. Proceedings of the 28th Annual ACM Symposium on Theory of Computing, 1996. <https://arxiv.org/abs/quant-ph/9605043>.
- [7] P. Høyer M. Boyer, G. Brassard and A. Tapp. Tight bounds on quantum searching. Fortschritte Der Physik, 1996. <https://arxiv.org/abs/quant-ph/9605034>.
- [8] Sawerwain Marek Wiśniewska Joanna. *Informatyka kwantowa. Wybrane obwody i algorytmy*. Wydawnictwo Naukowe PWN, 1 edition, 2015.
- [9] Wolfram MathWorld. Complete graph. <https://mathworld.wolfram.com/CompleteGraph.html>. Accessed: 04.08.2026.